distribution over the actions, and we can sample from this distribution to select actions. If a probability distribution is an unfamiliar concept to you, don't worry. We'll discuss it more in this chapter and throughout the book.

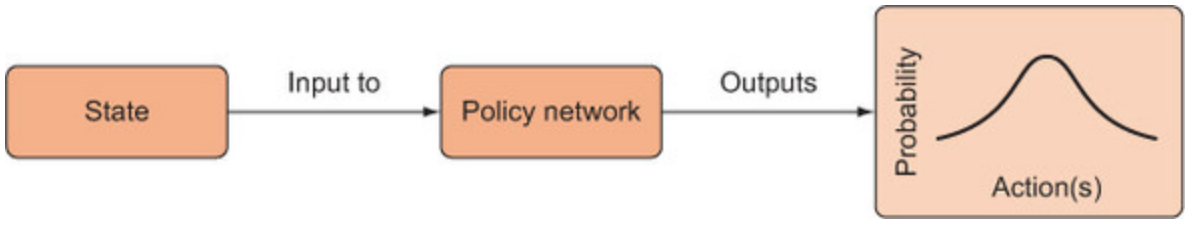
## 4.1. Policy function using neural networks

In this chapter we'll introduce a class of algorithms that allow us to approximate the policy function, $\pi(s)$, instead of the value function, $V_\pi$ or $Q$. That is, instead of training a network that outputs action values, we will train a network to output (the probability of) actions.

### 4.1.1. Neural network as the policy function

In contrast to a Q-network, a policy network tells us exactly what to do given the state we're in. No further decisions are necessary. All we need to do is randomly sample from the probability distribution $P(A|S)$, and we get an action to take (figure 4.2). The actions that are most likely to be beneficial will have the highest chance of being selected from random sampling, since they are assigned the highest probability.

**Figure 4.2. A policy network is a function that takes a state and returns a probability distribution over the possible actions.**




Imagine the probability distribution $P(A|S)$ as a jar filled with little notes with an action written on each. In a game with four possible actions, there will be notes with labels 1–4 (or 0–3 if they're indices in Python). If our policy network predicts that action 2 is the most likely to result in the highest reward, it will fill this jar with a lot of little notes labeled 2, and fewer notes labeled 1, 3, and 4. In order to select an action then, all we do is close our eyes and grab a random note from the jar. We're most likely to